Plan de reparto de trabajo: bot de notificaciones Telegram

1. Objetivo y arquitectura

Implementar un bot automatizado para aproximadamente 30–50 integrantes, utilizando:

- Django para la lógica del bot y su integración con el sistema.
- PostgreSQL existente para vínculos, eventos y entregas.
- Telegram para comandos y mensajes privados.
- Cloudflare gratuito para activar periódicamente el procesamiento.

Cada usuario realizará la vinculación inicial una vez. Después recibirá automáticamente los avisos que le correspondan.

La implementación buscará costo mensual adicional cero, sujeto a la capacidad y condiciones del alojamiento existente.

2. Reparto general

Responsable	Área principal	Resultado esperado
Persona A — German	Motor del bot, persistencia, vinculación, entrega y despliegue	Puede recibir comandos y entregar trabajos de forma recuperable
Persona B — Segundo integrante	Eventos del sistema, permisos, recordatorios y contenido	Las operaciones reales producen avisos correctos para sus destinatarios

Ambas personas desarrollarán las pruebas de sus propias partes.

La separación será por archivos y responsabilidades. Se realizará una coordinación breve al inicio, una comprobación intermedia del contrato y una integración funcional al final.

3. Acuerdo inicial obligatorio

Antes de programar, ambos dedicarán una sesión breve a definir el contrato entre sus componentes.

Se acordarán estos puntos:

Contrato	Acuerdo necesario

Registro de eventos	Función que recibe tipo, recurso, identificador del acontecimiento y datos mínimos
Destinatarios	Representación de las cuentas institucionales que podrían recibir el aviso
Autorización final	Función que determina si una entrega sigue permitida
Contenido	Resultado que contiene texto y botones autorizados
Recordatorios	Función que genera trabajos vencidos hasta una fecha determinada
Cancelación	Forma de invalidar trabajos antiguos al cambiar o cancelar un recurso

Las firmas y ejemplos quedarán en un documento compartido. Los nombres definitivos se fijarán antes de empezar.

Regla principal: Persona B produce acontecimientos y decide permisos y contenido; Persona A conserva los trabajos y los entrega.

Los contratos permitirán pruebas con componentes simulados. Así ninguno tendrá que esperar a que el otro termine toda su implementación.

4. Persona A: motor del bot y operación

A1. Crear la estructura y persistencia

Crear el módulo `telegram_notifications` y sus modelos:

- Vínculo entre usuario institucional y chat Telegram.
- Ticket temporal de vinculación.
- Evento de notificación.
- Trabajo de entrega.

Incluir estados, índices, restricciones de unicidad, reserva temporal, intentos y próxima fecha de ejecución.

Persona A será la única responsable de las migraciones del nuevo módulo.

Entregable: persistencia probada y contrato de registro de eventos disponible para Persona B.

A2. Implementar vinculación y comandos

Implementar:

- Webhook Telegram y validación de su secreto.
- Reconocimiento de actualizaciones repetidas.
- `/start` y solicitud de vinculación.

- Aprobación administrativa restringida.
- Consentimiento.
- `/estado`, `/pausar`, `/reanudar` y `/desvincular`.

La aprobación deberá asociar una cuenta institucional con un chat verificado. No se solicitarán contraseñas por Telegram.

Entregable: una cuenta de prueba puede vincularse, pausar y revocar su recepción.

A3. Implementar cola y entrega

Construir el motor que:

1. Reserva trabajos pendientes.
2. Solicita a Persona B la autorización y el contenido actual.
3. Comprueba el estado del vínculo.
4. Envía mediante Telegram.
5. Registra aceptación, cancelación, reintento o resultado incierto.

Implementar límites por lote y duración, recuperación de reservas vencidas, espera progresiva y tratamiento de bloqueos y errores `429`.

Entregable: entrega recuperable, probada inicialmente con eventos y respuestas simulados.

A4. Implementar activación y despliegue

Agregar el endpoint técnico de procesamiento y el Worker programador de Cloudflare.

La operación de procesamiento invocará los componentes de recordatorios de Persona B y posteriormente ejecutará la cola.

Configurar secretos, interruptor del canal, diagnóstico, limpieza y procedimiento de respaldo.

Entregable: configuración de pruebas reproducible y guía de despliegue y recuperación.

A5. Pruebas a cargo de Persona A

Cubrir:

- Vinculación inválida, vencida o reutilizada.
- Comandos repetidos.
- Pausa y desvinculación.
- Reserva concurrente y recuperación.
- Fallas de red, límites Telegram y bloqueo.
- Resultado incierto.
- Rechazo de llamadas no autorizadas al procesador.

5. Persona B: integración con el sistema y reglas de notificación

B1. Definir tipos de evento y plantillas

Crear el catálogo de acontecimientos y sus plantillas.

Definir qué cambios son relevantes y qué datos mínimos puede incluir cada mensaje. No se generará un aviso por cualquier modificación irrelevante.

Entregable: catálogo documentado, ejemplos y plantillas comprobadas.

B2. Conectar operaciones de negocio

Incorporar productores de eventos en los flujos de:

- Reuniones: creación, cambios relevantes, invitados y cancelación.
- Votaciones: programación, apertura y cierre.
- Proyectos: cambios relevantes de estado, publicación, archivo y asignaciones.
- Documentos: carga, cambios relevantes y retirada.
- Minutas y acuerdos: aprobación, asignación y cambios de estado.

Cada productor utilizará el contrato creado por Persona A.

La operación de negocio y su intención de notificación se guardarán dentro de la misma transacción. No se harán llamadas a Telegram desde estos productores.

Entregable: operaciones API reales que generan los eventos esperados.

B3. Implementar destinatarios y autorización

Construir las reglas que determinan:

- Quién podría recibir cada acontecimiento.
- Si conserva autorización justo antes del envío.
- Qué contenido mínimo puede recibir.
- Cuándo se descarta una entrega.

Reutilizar los selectores de acceso existentes.

Para votos pendientes, comprobar elegibilidad y ausencia de voto. Para documentos, validar el documento y su recurso padre.

Los responsables se resolverán por relaciones con cuentas y miembros; no por coincidencia del campo libre `responsible_name`.

Entregable: políticas probadas con miembros de comisiones distintas y pérdida de permisos.

B4. Implementar recordatorios y actualizaciones

Construir la generación de recordatorios para:

- Reuniones próximas.
- Votaciones próximas a cerrar.
- Acuerdos o seguimientos próximos a vencer.

Implementar invalidación por reprogramación, cancelación, cumplimiento y cambio de destinatarios.

Coordinar la ejecución explícita del ciclo de votaciones mediante los servicios existentes, conservando sus reglas.

Entregable: recordatorios correctos y repetibles sin crear trabajos duplicados.

B5. Pruebas a cargo de Persona B

Cubrir:

- Operación fallida sin evento de éxito.
- Cambio irrelevante sin aviso.
- Destinatarios correctos.
- Acceso cruzado entre comisiones.
- Reunión reprogramada o cancelada.
- Recordatorio de voto suprimido después de votar.
- Documento sin acceso vigente.
- Ejecuciones repetidas del generador de recordatorios.

6. Propiedad de archivos

La siguiente distribución es propuesta y se ajustará a las reglas del repositorio:

Archivos o área	Responsable
Modelos y migraciones de `telegram_notifications`	Persona A
Webhook, comandos, tickets y vinculación	Persona A
Cola, cliente Telegram y procesador	Persona A
Rutas, configuración y secretos del canal	Persona A
Worker Cloudflare	Persona A
Productores en módulos de negocio	Persona B
Políticas de destinatarios y autorización	Persona B

Plantillas y catálogo de eventos	Persona B
Generadores de recordatorios	Persona B
Pruebas	Cada persona sobre su componente

Persona B tendrá archivos propios dentro del nuevo módulo, por ejemplo `policies.py`, `templates.py`, `reminders.py` y `producers.py`. Persona A tendrá archivos separados para vinculación, recepción y entrega.

Los nombres de módulos evitarán importaciones circulares. Los modelos no importarán productores ni políticas de negocio.

Cualquier migración adicional necesaria se coordinará con Persona A y con el responsable del dominio.

7. Ramas y puntos de integración

Crear dos ramas desde el mismo commit actualizado de `develop`:

- `feature/telegram-engine`: Persona A.
- `feature/telegram-events`: Persona B.

Registrar el SHA de partida y abrir PR hacia la base de integración acordada.

El trabajo tendrá tres puntos de coordinación:

1. **Inicio:** fijar contratos y propiedad de archivos.
2. **Intermedio:** compartir modelos e interfaces estables de Persona A y comprobar un evento completo.
3. **Final:** integrar el catálogo y realizar pruebas completas.

La comprobación intermedia será pequeña: una operación API de prueba deberá producir un evento que el motor pueda procesar. Esto permite detectar incompatibilidades antes de terminar todos los dominios.

8. Equilibrio de carga y orden de implementación

La distribución inicial reserva aproximadamente la mitad del esfuerzo para cada persona. La estimación deberá revisarse al inspeccionar el código.

Persona A	Persona B
Persistencia e interfaces	Catálogo y plantillas
Vinculación y comandos	Productores y transacciones

Cola y recuperación	Destinatarios y permisos
Programador y operación	Recordatorios y cancelaciones
Pruebas técnicas	Pruebas de negocio

Ambos comenzarán con reuniones y votaciones. Después ampliarán documentos, proyectos, minutas y acuerdos usando la misma estructura.

Noticias y eventos institucionales se incorporarán cuando se acuerde su fuente oficial. «Propuestas» requiere confirmar su significado antes de crear productores.

Si una parte resulta considerablemente mayor, se transferirán tareas completas con archivos propios, como documentación o pruebas funcionales, evitando que ambos editen simultáneamente el mismo componente.

9. Validación final conjunta

Ambos comprobarán en un entorno separado:

1. Vinculación de dos cuentas de prueba.
2. Creación de una reunión mediante API.
3. Recepción únicamente en el chat autorizado.
4. Reprogramación y cancelación sin recordatorios antiguos.
5. Recordatorio de votación solo a quien aún no votó.
6. Recuperación tras interrumpir el procesamiento.
7. Revocación de acceso antes del envío.
8. Consumo y latencia con una audiencia equivalente a 50 personas.
9. Respaldo y restauración sin envíos accidentales.

Persona A verificará la operación y entrega. Persona B verificará el acontecimiento, la audiencia y el contenido.

La activación institucional se realizará después de revisar las migraciones, completar las pruebas y confirmar las cuotas de infraestructura. Las operaciones del frontend que todavía sean locales se mantendrán identificadas como dependencias de integración.